Scanned Code Report

AUDITAGENT

Powered by NETHERMIND SECURITY

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Developer Scan

Scan ID 7	Date June 23, 2026
Organization arkhai-io	Repository alkahest
Branch dev	Commit Hash 4d3f9043...4913213c

Contracts in scope

contracts/src/ArbiterUtils.sol
contracts/src/BaseAttester.sol
contracts/src/BaseEscrowObligation.sol
contracts/src/BaseEscrowObligationUnconditional.sol
contracts/src/BaseObligation.sol
contracts/src/IArbiter.sol
contracts/src/IEscrow.sol
contracts/src/SchemaRegistryUtils.sol
contracts/src/arbiters/ERC8004Arbiter.sol
contracts/src/arbiters/IntrinsicsArbiter.sol
contracts/src/arbiters/ReferencesEscrowArbiter.sol
contracts/src/arbiters/TrivialArbiter.sol
contracts/src/arbiters/TrustedOracleArbiter.sol
contracts/src/arbiters/attestation-properties/AttesterArbiter.sol
contracts/src/arbiters/attestation-properties/ExpirationTimeAfterArbiter.sol
contracts/src/arbiters/attestation-properties/ExpirationTimeBeforeArbiter.sol
contracts/src/arbiters/attestation-properties/ExpirationTimeEqualArbiter.sol
contracts/src/arbiters/attestation-properties/RecipientArbiter.sol
contracts/src/arbiters/attestation-properties/RefUidArbiter.sol
contracts/src/arbiters/attestation-properties/RevocableArbiter.sol
contracts/src/arbiters/attestation-properties/SchemaArbiter.sol
contracts/src/arbiters/attestation-properties/TimeAfterArbiter.sol
contracts/src/arbiters/attestation-properties/TimeBeforeArbiter.sol
contracts/src/arbiters/attestation-properties/TimeEqualArbiter.sol
contracts/src/arbiters/attestation-properties/UidArbiter.sol
contracts/src/arbiters/confirmation/ExclusiveRevocableConfirmationArbiter.sol
contracts/src/arbiters/confirmation/ExclusiveUnrevocableConfirmationArbiter.sol
contracts/src/arbiters/confirmation/NonexclusiveRevocableConfirmationArbiter.sol
contracts/src/arbiters/confirmation/NonexclusiveUnrevocableConfirmationArbiter.sol
contracts/src/arbiters/logical/AllArbiter.sol
contracts/src/arbiters/logical/AnyArbiter.sol
contracts/src/obligations/CommitRevealObligation.sol
contracts/src/obligations/StringObligation.sol
contracts/src/obligations/escrow/default/AttestationEscrowObligation.sol
contracts/src/obligations/escrow/default/AttestationReferenceEscrowObligation.sol
contracts/src/obligations/escrow/default/ERC1155EscrowObligation.sol
contracts/src/obligations/escrow/default/ERC20EscrowObligation.sol
contracts/src/obligations/escrow/default/ERC721EscrowObligation.sol

contracts/src/obligations/escrow/default/NativeTokenEscrowObligation.sol

contracts/src/obligations/escrow/default/TokenBundleEscrowObligation.sol

contracts/src/obligations/escrow/hook-based/HookEscrowObligation.sol

contracts/src/obligations/escrow/hook-based/HooksEscrowObligation.sol

contracts/src/obligations/escrow/hook-based/IEscrowHook.sol

contracts/src/obligations/escrow/hook-based/hooks/AttestationEscrowHook.sol

contracts/src/obligations/escrow/hook-based/hooks/AttestationReferenceEscrowHook.sol

contracts/src/obligations/escrow/hook-based/hooks/ERC1155EscrowHook.sol

contracts/src/obligations/escrow/hook-based/hooks/ERC20EscrowHook.sol

contracts/src/obligations/escrow/hook-based/hooks/ERC721EscrowHook.sol

contracts/src/obligations/escrow/hook-based/hooks/NativeTokenEscrowHook.sol

contracts/src/obligations/escrow/unconditional/UnconditionalAttestationEscrowObligation.sol

contracts/src/obligations/escrow/unconditional/UnconditionalAttestationReferenceEscrowObligation.sol

contracts/src/obligations/escrow/unconditional/UnconditionalERC1155EscrowObligation.sol

contracts/src/obligations/escrow/unconditional/UnconditionalERC20EscrowObligation.sol

contracts/src/obligations/escrow/unconditional/UnconditionalERC721EscrowObligation.sol

contracts/src/obligations/escrow/unconditional/UnconditionalNativeTokenEscrowObligation.sol

contracts/src/obligations/escrow/unconditional/UnconditionalTokenBundleEscrowObligation.sol

contracts/src/obligations/payment/ERC1155PaymentObligation.sol

contracts/src/obligations/payment/ERC20PaymentObligation.sol

contracts/src/obligations/payment/ERC721PaymentObligation.sol

contracts/src/obligations/payment/NativeTokenPaymentObligation.sol

contracts/src/obligations/payment/TokenBundlePaymentObligation.sol

contracts/src/utils/atomic/AtomicAttestationUtils.sol

contracts/src/utils/atomic/AtomicPaymentUtils.sol

contracts/src/utils/splitters/BaseSplitter.sol

contracts/src/utils/splitters/ERC1155Splitter.sol

contracts/src/utils/splitters/ERC20Splitter.sol

contracts/src/utils/splitters/NativeTokenSplitter.sol

contracts/src/utils/splitters/SplitterVerification.sol

contracts/src/utils/splitters/TokenBundleSplitter.sol

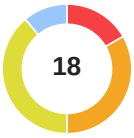
contracts/src/utils/splitters/TokenBundleSplitterBase.sol

contracts/src/utils/splitters/TokenBundleSplitterUnvalidated.sol

Code Statistics

Findings	Contracts Scanned	Lines of Code
18	71	8110

Findings Summary



Total Findings

- High Risk (3)
- Medium Risk (6)
- Low Risk (7)
- Info (2)
- Best Practices (0)

Code Summary

The protocol provides a modular decentralized obligation and escrow framework built on top of the Ethereum Attestation Service (EAS). It allows users to lock assets—including Native tokens, ERC20, ERC721, ERC1155, or mixed bundles—behind programmable fulfillment conditions. These conditions are evaluated by an extensible arbitration system. The architecture revolves around representing the terms of an escrow as an EAS attestation, ensuring transparency and cryptographic verifiability.

The system is highly composable, featuring several key components:
- **Escrows & Obligations**: Asset-specific contracts that manage the locking, releasing, and reclaiming of funds based on EAS attestations.
- **Arbiters**: Modular validation contracts that evaluate if a fulfillment attestation meets an escrow's demand. They support diverse logic, including intrinsic attestation properties (e.g., expiration, schema, recipient), logical combinations (`AnyArbiter`, `AllArbiter`), and trusted oracle decisions.
- **Hooks**: Custom logic attachments that can be executed during the lock, release, or return phases of the escrow lifecycle, allowing for flexible asset management without altering the core escrow logic.
- **Splitters**: specialized utility contracts that collect escrowed assets and distribute them among multiple recipients according to off-chain or oracle-provided split decisions.
- **Commit-Reveal & Atomic Utilities**: Features to prevent front-running via bonded commitments, as well as helper contracts to execute multi-step operations (like paying a bundle and collecting an escrow) in a single atomic transaction.

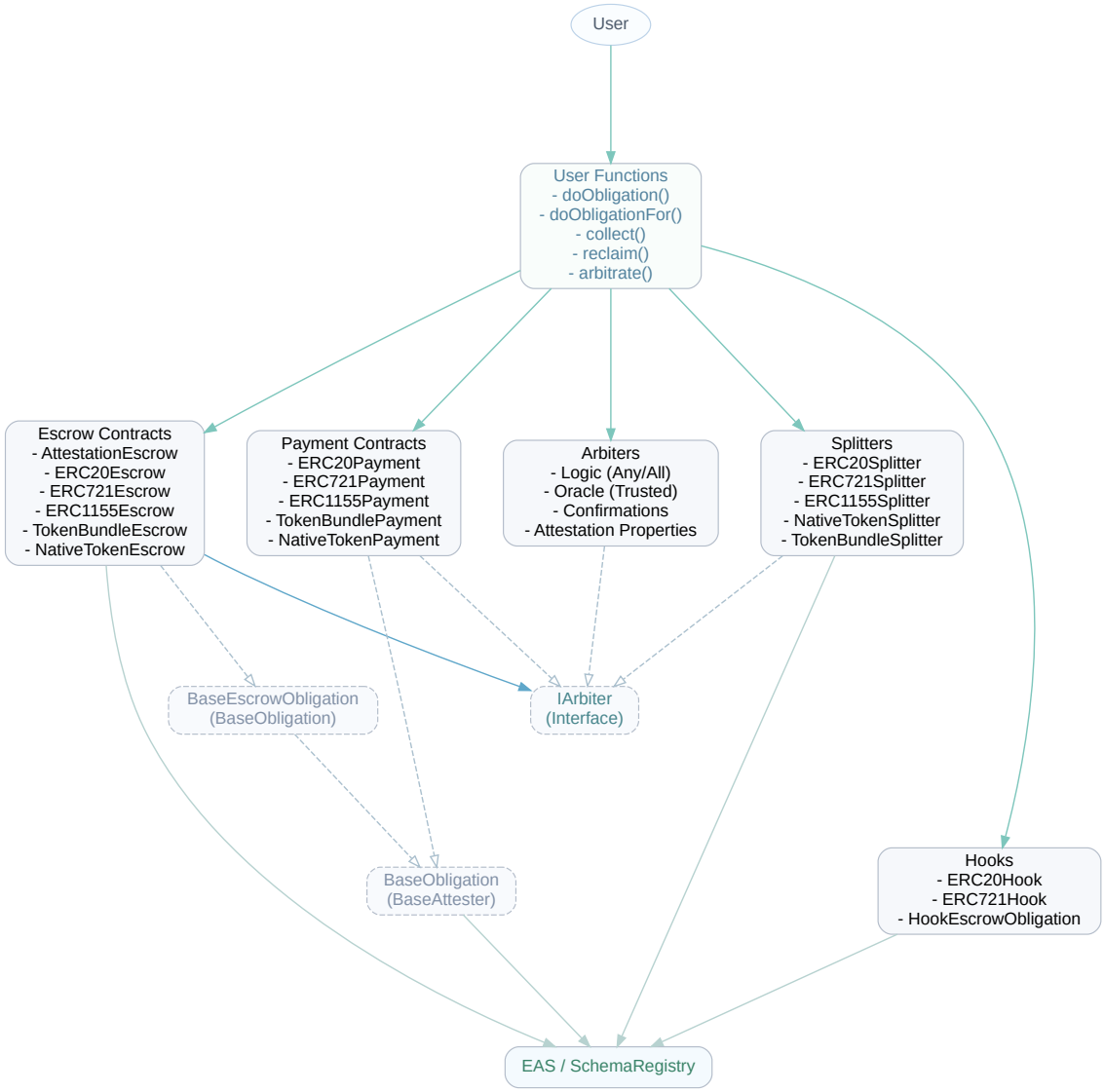
Main Entry Points and Actors

- `doObligation` / `doObligationFor`: Used by Escrowers/Payers to lock assets and create an obligation or escrow attestation.
- `collect`: Used by Fulfillers to claim escrowed assets by providing a valid fulfillment attestation UID that satisfies the arbiter.
- `reclaim`: Used by Escrowers to recover their locked assets if an escrow attestation has expired without fulfillment.
- `commit`: Used by Fulfillers to lock a native-token bond and a commitment hash to prevent front-running on certain obligations.
- `revealAndCollect`: Used by Fulfillers to reveal a previously committed payload, create a fulfillment attestation, and atomicaly collect the target escrow.
- `slashBond`: Used by any User to slash the bond of a committer who failed to reveal their obligation payload before the deadline.
- `arbitrate`: Used by Trusted Oracles to record a cryptographic decision regarding a specific fulfillment and escrow context.
- `requestArbitration`: Used by Attesters or Recipients to emit a request for an oracle to review a specific fulfillment.
- `confirm` / `revoke`: Used by Escrow Recipients (acting as arbiters) to explicitly approve or reject a fulfillment in confirmation-based arbiters.
- `collectAndDistribute` / `unsafePartiallyCollectAndDistribute`: Used by any User to execute the collection of an escrow and immediately process the distribution of assets via a Splitter contract.
- `payErc20AndCollect` / `payNativeAndCollect` / `payBundleAndCollect`: Used by Fulfillers to pay an expected demand and immediately collect the corresponding escrow atomically.
- `permitAndPayWithErc20` / `permitAndPayErc20AndCollect` / `permitAndPayBundleAndCollect`: Used by Fulfillers to utilize EIP-2612 signatures to approve token transfers, pay the obligation, and collect the escrow in

one transaction.

- `attestAndCreateReferenceEscrow` / `attestAndCreateUnconditionalReferenceEscrow`: Used by Users to create a new EAS attestation and simultaneously wrap it in an attestation-reference escrow.

Code Diagram



1 of 18 Findings

contracts/src/obligations/escrow/hook-based/hooks/ERC20EscrowHook.sol

contracts/src/obligations/escrow/hook-based/hooks/ERC721EscrowHook.sol

contracts/src/obligations/escrow/hook-based/hooks/ERC1155EscrowHook.sol

Permissionless token hooks let arbitrary callers steal any ERC20/ERC721/ERC1155 previously approved to the hook

High Risk

The token-backed hooks are externally callable and do not authenticate who is allowed to initiate a lock. Each `onLock` trusts the caller-supplied `from` address, performs a token transfer using the hook contract's own allowance/operator approval, and then credits the resulting deposit to `msg.sender`, not to `from` and not to a verified escrow contract. For example, the ERC20 hook executes `IERC20(decoded.token).trySafeTransferFrom(from, address(this), decoded.amount);` and then records `deposits[msg.sender][decoded.token] += decoded.amount;`. The ERC721 and ERC1155 hooks follow the same pattern.

Because normal usage requires users to approve the hook contract directly, any attacker can exploit that approval without interacting with the escrow obligations at all. The attacker calls `onLock` with `from = victim`, causing the hook to pull the victim's approved assets into itself, and the hook records the deposit under the attacker's address because `msg.sender` is the attacker. The attacker can then immediately call `onRelease` or `onReturn` to transfer the assets to any destination. The `escrow` parameter is ignored, there is no allowlist of valid obligation contracts, and `onRelease` / `onReturn` do not verify that a real escrow lifecycle is being executed.

This turns every approval granted to these hooks into a standing theft vector. A victim does not need to have created an escrow yet; approving the hook in preparation for future use is sufficient. The issue affects fungible, non-fungible, and semi-fungible assets.

2 of 18 Findings

contracts/src/utils/splitters/ERC20Splitter.sol
contracts/src/utils/splitters/ERC1155Splitter.sol
contracts/src/utils/splitters/NativeTokenSplitter.sol
contracts/src/utils/splitters/TokenBundleSplitterBase.sol
contracts/src/utils/splitters/TokenBundleSplitter.sol
contracts/src/utils/splitters/TokenBundleSplitterUnvalidated.sol

Splitter collection paths do not verify that the splitter is the escrow's configured arbiter

High Risk

All splitter collection flows validate only that the escrow attestation was emitted by the supplied escrow contract and uses that contract's schema, then immediately decode the escrow's stored demand as splitter demand and call `collect`. None of them verify that the escrow itself actually selected the splitter as its arbiter.

The relevant pattern is:

```
Attestation memory escrowAttestation = eas.getAttestation(escrow);\nescrowAttestation.verifyEscrowAttesta
```

There is no check equivalent to `escrowData.arbiter == address(this)` before the escrow is collected. As a result, any same-type escrow whose attestation data is ABI-compatible with the splitter's `EscrowObligationData` can be collected through the splitter even when that escrow was governed by a completely different arbiter.

This breaks asset routing in two harmful ways. First, if the foreign escrow's demand bytes are also decodable as the splitter's `DemandData` and the decoded `oracle` is attacker-controlled, the attacker can pre-store arbitrary splits and use the splitter as a public drain path for an escrow that never delegated authority to that splitter. Second, even without a matching decision, the call can still succeed after `IEscrow.collect(...)`, with `splits.length == 0`, leaving the collected assets stranded inside the splitter after the underlying escrow has been irreversibly consumed. Because the escrow has already been collected, the original parties cannot simply retry through the correct path.

Severity Note:
- IEscrow.collect transfers the escrowed assets to the caller (the splitter) upon successful collection OR otherwise routes the assets such that verifyDelta passes only when the splitter receives the funds.
- IEscrow.collect does not itself restrict the caller to the escrow’s configured arbiter (or the escrow’s design allows any caller to trigger collection with assets delivered to the caller).

3 of 18 Findings

contracts/src/utils/atomic/AtomicPaymentUtils.sol

AtomicPaymentUtils trusts arbitrary attesters as escrows and accepts any non-reverting `collect` as success

High Risk

`AtomicPaymentUtils` does not verify that the supplied attestation is a genuine protocol escrow before paying its decoded demand. `_getEscrowAndDemand` only checks that an attestation exists, then treats `escrow.attester` as an arbitrary condition decoder:

```
escrow = eas.getAttestation(escrowUid);\nif (escrow.uid == bytes32(0) || escrow.uid != escrowUid) revert
```

After creating the payment fulfillment, `_collect` simply calls the same arbitrary attester as `IEscrow` and considers any non-reverting return value to be a successful collection:

```
try IEscrow(escrowContract).collect(escrowUid, fulfillmentUid) {}\ncatch { revert CouldntCollectEscrow()
```

There is no validation that the attestation was emitted by an expected escrow contract, that it uses an expected escrow schema, or that any escrowed asset balance/ownership actually moved as a consequence of `collect`. A malicious contract can therefore mint an EAS attestation with itself as the attester, implement `decodeCondition` so that the helper pays attacker-chosen terms, and implement `collect` to do nothing except return success. When a victim calls any `*AndCollect` helper against that UID, the helper transfers the victim's assets into the payment obligation flow and then finishes successfully without any real escrow ever being released.

This affects `payErc20AndCollect`, `permitAndPayErc20AndCollect`, `payErc721AndCollect`, `payErc1155AndCollect`, `payNativeAndCollect`, `payBundleAndCollect`, and `permitAndPayBundleAndCollect`.

Severity Note:
- Integrators or users may submit escrow UIDs without verifying the attester is an approved escrow contract or schema.
- EAS is permissionless, allowing arbitrary attesters to mint such attestations.
- The PaymentObligation.doObligationFor paths effect actual transfers to the specified payee (or otherwise move control of assets away from the victim).
- A malicious collect() can be a no-op and return success.

4 of 18 Findings contracts/src/obligations/escrow/default/TokenBundleEscrowObligation.sol

`unsafePartiallyCollectEscrow` incorrectly restricts the caller to `escrow.recipient` instead of `fulfillment.recipient`

Medium Risk

The `TokenBundleEscrowObligation` contract includes an `unsafePartiallyCollectEscrow` function designed to be used as a 'last resort' when some tokens in a bundle are stuck (e.g., paused or blocklisted). It allows an escrow to be collected partially, continuing even if individual token transfers fail. Because this function forces the recipient to accept partial losses of assets, it must be restricted to the party who is receiving the assets to prevent third parties from forcing them to accept a partial payment.

The intended receiver of a successful escrow collection is the Fulfiller (`fulfillment.recipient`). However, the function incorrectly requires the caller to be `escrow.recipient`:

```
if (msg.sender != escrow.recipient) revert UnauthorizedCall();
```

In this protocol, `escrow.recipient` is the Escrower (the depositor of the funds), because `reclaim` uses `escrow.recipient` as the refund address when an escrow expires.

This incorrect access control check leads to two severe issues:
1. The Fulfiller cannot call `unsafePartiallyCollectEscrow` to salvage their payment, rendering the function completely useless for its intended beneficiary.
2. The Escrower is the only one who can call it. The Escrower has no incentive to give the Fulfiller a partial payment because they can simply wait for the escrow to expire and call `unsafePartiallyReclaimExpired` to recover the unstuck tokens for themselves. Furthermore, if the Escrower does call it, they force the Fulfiller to take a partial loss without their consent, even if the Fulfiller would have preferred to wait for the token to become transferable again.

To fix this issue, the access control check should verify that the caller is `fulfillment.recipient` so that the party suffering the loss is the one authorizing the partial collection.

Severity Note:
- There exists an atomic collect path in the base contract that reverts if any transfer fails (requiring the partial path when assets are stuck).
- Escrows often include an expiry after which the escrower can reclaim.
- At least some real tokens can be paused/blacklisted or otherwise fail transfers, making the partial path relevant.

5 of 18 Findings

contracts/src/utils/splitters/BaseSplitter.sol

contracts/src/obligations/payment/NativeTokenPaymentObligation.sol

contracts/src/obligations/payment/TokenBundlePaymentObligation.sol

Splitter fulfillment helper refunds away native payments while still minting valid payment attestations

Medium Risk

`BaseSplitter.createFulfillment` attempts to protect callers from accidentally leaving ETH in the splitter by refunding any post-call balance increase back to the external caller. That refund logic is unsafe when the obligation being created is itself a native-payment obligation that legitimately transfers ETH to the splitter during attestation creation.

The relevant flow is:

```
function createFulfillment(address obligationContract, bytes calldata data, uint64
expirationTime, bytes32 refUID)
    external
    payable
    nonReentrant
    returns (bytes32 fulfillmentUid)
{
    uint256 retainedBalance = address(this).balance - msg.value;
    fulfillmentUid = IObligation(obligationContract).doObligationRaw{value: msg.value}(data,
expirationTime, refUID);
    _validateCreatedFulfillment(fulfillmentUid, obligationContract, data, expirationTime,
refUID);
    if (fulfillers[fulfillmentUid] != address(0)) revert
FulfillerAlreadyRecorded(fulfillmentUid);
    fulfillers[fulfillmentUid] = msg.sender;
    _refundNativeBalanceIncrease(retainedBalance, msg.sender);
}

function _refundNativeBalanceIncrease(uint256 retainedBalance, address recipient) internal {
    uint256 currentBalance = address(this).balance;
    if (currentBalance <= retainedBalance) return;

    uint256 refundAmount = currentBalance - retainedBalance;
    (bool success,) = payable(recipient).call{value: refundAmount}("");
    if (!success) revert NativeTokenRefundFailed(recipient, refundAmount);
}
```

When the chosen `obligationContract` is `NativeTokenPaymentObligation`, attestation creation sends the supplied ETH to the encoded payee before the attestation is emitted:

```
(bool success,) = payable(obligationData.payee).call{value: obligationData.amount}("");
if (!success) {
    revert NativeTokenTransferFailed(obligationData.payee, obligationData.amount);
}
```

If that payee is the splitter itself, the ETH returns to the splitter during `doObligationRaw`. Control then returns to `createFulfillment`, which treats that returned payment as an accidental balance increase and refunds it to the external caller. The attestation remains valid and `_validateCreatedFulfillment` still passes, so the caller receives a bona fide fulfillment UID while recovering the full payment amount. The same issue exists for the native component of `TokenBundlePaymentObligation`.

As a result, any workflow that relies on `BaseSplitter.createFulfillment` to produce splitter-owned fulfillments for payment-based obligations can be bypassed whenever the payment route sends native ETH back into the splitter during creation. The fulfillment attestation states that payment occurred, but the caller can end the transaction with their ETH refunded and later use that attestation to satisfy escrow conditions and unlock third-party assets without bearing the intended native payment cost.

Severity Note:
- An escrow/oracle releases assets based solely on the presence of a valid payment attestation referencing the escrow UID and specifying the splitter as payee.
- Integrations actually configure the obligation to pay native ETH to the splitter during fulfillment creation.
- No additional off-chain/on-chain checks validate net payment to the splitter beyond the attestation itself.

6 of 18 Findings

contracts/src/obligations/CommitRevealObligation.sol

contracts/src/utils/splitters/BaseSplitter.sol

Reveal is not bound to the original committer, allowing front-run griefing and breaking sentinel-based splitter payouts

Medium Risk

`CommitRevealObligation` validates only that a revealed attestation matches an existing commitment hash. It never requires the revealer to be the address that originally posted the bond. In `_afterAttest`, the contract reconstructs the commitment from `attestation.refUID`, `attestation.recipient` and `attestation.data`, loads `CommitInfo`, and only checks existence, age, deadline, and whether the bond was already claimed:

```
bytes32 revealedCommitment =
    keccak256(abi.encode(attestation.refUID, attestation.recipient,
keccak256(attestation.data)));

CommitInfo memory info = commitments[revealedCommitment];
...
if (commitmentClaimed[revealedCommitment]) {
    revert BondAlreadyClaimed(revealedCommitment);
}

commitmentClaimed[revealedCommitment] = true;
(bool success,) = info.committer.call{value: amount}("");
```

Because `msg.sender` is never compared against `info.committer`, any observer who sees a pending reveal transaction can copy the same `(data, recipient, refUID)` and submit it first. The bond is refunded to the original committer, but the original reveal transaction is then forced to revert because the commitment has already been consumed.

This becomes materially harmful when the intended recipient is a splitter contract. `BaseSplitter` uses `fulfillers[fulfillmentUid]` to resolve `EXECUTOR_SENTINEL`, but that mapping is populated only when the fulfillment is created through `BaseSplitter.createFulfillment`. A frontrunner can bypass that helper and call `CommitRevealObligation.doObligationFor(..., recipient=splitter, refUID)` directly, creating a valid splitter-addressed fulfillment without recording a fulfiller. Any later split decision that uses `EXECUTOR_SENTINEL` will revert in `_resolveSentinel` with `NoFulfillerRecorded`, so the escrow can no longer be distributed through the splitter path even though the commitment has been consumed and the fulfillment is valid.

This defeats the stated anti-front-running purpose of the commitment scheme for reveal-time mempool races and can permanently break executor-dependent distribution flows.

Severity Note:
- Oracle decisions commonly reference the fulfillment created by the (front-run) reveal.
- At least some splitter decisions include EXECUTOR_SENTINEL, making missing fulfiller mapping a hard revert.
- There is no alternative path in production that bypasses _resolveSentinel for these decisions.

7 of 18 Findings

contracts/src/arbiters/attestation-properties/ExpirationTimeAfterArbiter.sol

contracts/src/arbiters/attestation-properties/ExpirationTimeBeforeArbiter.sol

Incorrect handling of EAS `expirationTime == 0` sentinel value allows valid fulfillments to be rejected and invalid ones accepted

Medium Risk

In the Ethereum Attestation Service (EAS), an `expirationTime` of `0` is a sentinel value indicating that an attestation never expires (i.e., its expiration is effectively at infinity). The `ExpirationTimeAfterArbiter` and `ExpirationTimeBeforeArbiter` contracts perform direct integer comparisons on the expiration timestamp without accounting for this sentinel value.

In `ExpirationTimeAfterArbiter`:

```
if (fulfillment.expirationTime < demand_.expirationTime) {
    revert ExpirationTimeNotAfter();
}
```

If an escrow demands that a fulfillment expires at or after a specific timestamp (e.g., `demand_.expirationTime > 0`), and a fulfiller provides an attestation that never expires (`fulfillment.expirationTime == 0`), the condition evaluates to `0 < demand_.expirationTime` which is `true`. The arbiter improperly rejects the non-expiring attestation, even though an attestation that never expires effectively satisfies any "expires after X" constraint.

In `ExpirationTimeBeforeArbiter`:

```
if (fulfillment.expirationTime > demand_.expirationTime) {
    revert ExpirationTimeNotBefore();
}
```

If a fulfillment attestation never expires (`fulfillment.expirationTime == 0`), the condition evaluates to `0 > demand_.expirationTime` which is `false`. The arbiter improperly accepts the attestation, completely bypassing the demand that the fulfillment must expire before a specific time.

This flaw allows invalid fulfillments to bypass the expected constraints and causes valid non-expiring fulfillments to be rejected, breaking the core logical guarantees of these temporal arbiters.

Severity Note:
- These arbiters gate escrow release or fulfillment success in a flow where passing check() can lead to value movement.
- Attestations with expirationTime==0 exist and can be chosen by fulfillers (e.g., self-attested or from permissive schemas), or trusted attesters may legitimately issue non-expiring attestations.
- No other arbiter in the composition corrects this specific 0-sentinel misinterpretation.

8 of 18 Findings contracts/src/arbiters/attestation-properties/ExpirationTimeBeforeArbiter.sol

`ExpirationTimeBeforeArbiter` treats EAS's `expirationTime == 0` no-expiration sentinel as an earlier timestamp, letting perpetual fulfillments satisfy an upper-bound expiry check. (github.com)

Medium Risk

EAS documents `expirationTime` as a Unix timestamp where `0` means "no expiration," not "January 1, 1970." (github.com) `ExpirationTimeBeforeArbiter` compares the raw integer value directly:

```
if (fulfillment.expirationTime > demand_.expirationTime) { revert ExpirationTimeNotBefore(); }
```

Because of that, any non-expiring attestation with `fulfillment.expirationTime == 0` will pass every positive cutoff `T`, since `0 > T` is false. The contract therefore accepts a perpetual fulfillment even when the escrow creator encoded a demand intended to require an attestation that expires on or before a specific time.

This is not a mere policy choice by the escrow creator; it is a semantic mismatch between the arbiter's advertised behavior ("expiration time is at or before a demanded timestamp") and EAS's sentinel-based representation of non-expiring attestations. A counterparty can intentionally mint a non-expiring fulfillment and satisfy a maximum-expiration constraint that was supposed to reject such attestations, causing settlement under looser conditions than the escrow creator specified.

Severity Note:
- The arbiter is part of an executable escrow path where a passing check contributes to releasing funds.
- The issuer and other attestation properties are otherwise acceptable so the only bypass needed is the expiration constraint.
- No additional arbiter or business rule separately rejects non-expiring attestations.

9 of 18 Findings

contracts/src/utils/splitters/TokenBundleSplitterBase.sol

contracts/src/utils/splitters/TokenBundleSplitter.sol

contracts/src/utils/splitters/TokenBundleSplitterUnvalidated.sol

Duplicate ERC20 entries or duplicate ERC1155 `(token,tokenId)` entries make bundle splitters unable to collect a valid escrow

Medium Risk

`TokenBundleSplitterBase` snapshots balances per array index and then verifies each post-collection delta independently. That logic assumes every ERC20 entry refers to a unique token address and every ERC1155 entry refers to a unique `(token, tokenId)` pair. The contract does not enforce those uniqueness assumptions.

The relevant code is:

```
function _erc20Balances(EscrowObligationData memory escrowData) internal view returns
(uint256[] memory balances) {
    balances = new uint256[](escrowData.erc20Tokens.length);
    for (uint256 i; i < escrowData.erc20Tokens.length; ++i) {
        balances[i] = IERC20(escrowData.erc20Tokens[i]).balanceOf(address(this));
    }
}
...
for (uint256 i; i < escrowData.erc20Tokens.length; ++i) {
    SplitterVerification.verifyDelta(
        erc20Before[i], IERC20(escrowData.erc20Tokens[i]).balanceOf(address(this)),
escrowData.erc20Amounts[i]
    );
}
```

If the escrow bundle contains the same ERC20 token twice, both `erc20Before` snapshots record the same starting balance. After `collect`, the splitter balance increases by the **sum** of both escrowed amounts. The first `verifyDelta` call then observes the aggregate increase instead of the per-entry amount and reverts. The exact same failure mode exists for duplicate ERC1155 entries when both the token address and token id repeat, because `_erc1155Balances` snapshots the same slot twice and `_verifyCollectedDeltas` validates each duplicate entry against the full aggregate delta.

Example: an escrow with `erc20Tokens = [USDC, USDC]` and `erc20Amounts = [100, 50]` is internally consistent and can pass `TokenBundleSplitter._validateSplits`, but `_verifyCollectedDeltas` will compare a post-collect delta of `150` against `100` for the first index and revert. As a result, both `collectAndDistribute` and `unsafePartiallyCollectAndDistribute` fail for these bundles. If the fulfillment recipient is the splitter, a direct manual collection would send the assets to the splitter, but there is no working bundle-splitter path left to process them, leaving the escrow effectively stuck.

Severity Note:
- Escrow attestation data (including duplicate entries and recipient set to the splitter) is immutable post-creation.
- IEscrow.collect cannot be customized per-escrow after creation and the intended processing route is via the splitter.
- There is no alternate distribution function that skips _collectAndDecode().

10 of 18 Findings

contracts/src/obligations/escrow/default/TokenBundleEscrowObligation.sol

contracts/src/obligations/escrow/unconditional/UnconditionalTokenBundleEscrowObligation.sol

Silent ETH trapping in TokenBundleEscrowObligation when nativeAmount is zero

Low Risk

The `_lockEscrow` function in `TokenBundleEscrowObligation` and `UnconditionalTokenBundleEscrowObligation` processes mixed-asset bundle deposits. It is designed to strictly ensure that the `msg.value` sent exactly matches `decoded.nativeAmount`. However, the exact-match check is nested inside an `if (decoded.nativeAmount > 0)` block. If an escrow obligation is created with a `nativeAmount` of `0`, this conditional block evaluates to false, causing the contract to entirely bypass the `msg.value` validation.

Because the entry points (`doObligation` and `doObligationFor` in `BaseObligation` and `TokenBundleEscrowObligation`) are declared `payable` to accommodate bundles that do include native tokens, users can accidentally attach native ETH to the transaction even when `nativeAmount` is set to `0`. Instead of reverting due to an incorrect payment, the contract will silently accept the execution, locking the user's `msg.value` within the contract permanently, as there is no mechanism to refund excess or unexpected ETH in this escrow type.

```
function _lockEscrow(bytes memory data, address from) internal override {
    ObligationData memory decoded = abi.decode(data, (ObligationData));
    validateArrayLengths(decoded);

    // Handle native tokens
    if (decoded.nativeAmount > 0) {
        // ONLY validates msg.value if nativeAmount > 0
        if (msg.value != decoded.nativeAmount) {
            revert IncorrectPayment(decoded.nativeAmount, msg.value);
        }
    }

    // Handle token bundle
    transferInTokenBundle(decoded, from);
}
```

This breaks the consistency of strict payment enforcement seen in other areas of the protocol (such as `NativeTokenEscrowObligation`) and leads to a permanent loss of funds if the caller makes an error.

Severity Note:
- Some callers/integrations may accidentally include non-zero msg.value when nativeAmount is 0.

11 of 18 Findings | contracts/src/obligations/escrow/unconditional/UnconditionalTokenBundleEscrowObligation.sol

Zero-native bundle escrows accept and strand unexpected ETH

Low Risk

`UnconditionalTokenBundleEscrowObligation` is payable, but its payment validation only executes when the encoded bundle includes a non-zero native component:

```
if (decoded.nativeAmount > 0) {
    if (msg.value != decoded.nativeAmount) {
        revert IncorrectPayment(decoded.nativeAmount, msg.value);
    }
}
```

When `decoded.nativeAmount == 0`, the function performs no `msg.value` check at all and still proceeds to create the escrow attestation. The escrow state records `nativeAmount` as zero, so the extra ETH is not tracked anywhere in the attested obligation data. Later, both collection and reclaim only transfer `decoded.nativeAmount`:

```
if (decoded.nativeAmount > 0) {
    (bool success,) = payable(to).call{value: decoded.nativeAmount}("");
    if (!success) {
        revert NativeTokenTransferFailed(to, decoded.nativeAmount);
    }
}
```

As a result, any ETH accidentally or mistakenly sent alongside a token-only bundle is detached from the escrow lifecycle: it is neither released to the fulfiller nor returned on reclaim. The caller loses the ability to recover that surplus through normal protocol flows, while the contract balance becomes higher than the sum of native amounts actually represented by live escrows. This creates direct user fund loss for token-only bundle escrows and breaks the expectation that payable entrypoints enforce exact payment semantics.

12 of 18 Findings

contracts/src/obligations/escrow/hook-based/hooks/AttestationEscrowHook.sol

contracts/src/obligations/escrow/hook-based/hooks/AttestationReferenceEscrowHook.sol

Attestation hooks can be invoked directly to mint hook-issued attestations without any escrow lifecycle

Low Risk

Both attestation-based hooks rely on a local `pending[msg.sender][keccak256(data)]` counter as their only gate before issuing an attestation, but `onLock` and `onRelease` are completely unrestricted external entrypoints. Any account can create its own pending entry by calling `onLock` directly and can then immediately consume it by calling `onRelease` directly, without ever creating, fulfilling, or collecting an escrow.

In `AttestationEscrowHook`, the sequence is:
`onLock(...)` -> increment `pending[msg.sender][dataHash]` -> `onRelease(...)` -> decrement pending -> `eas.attest{value: requiredValue}(decoded.attestation)`.
Because `onRelease` does not verify that `msg.sender` is an approved escrow contract, and does not validate that the supplied escrow attestation corresponds to a real escrow flow, the hook contract itself becomes a public attestation proxy.

In `AttestationReferenceEscrowHook`, the problem is more severe because no ETH is required at lock time. Any caller can execute:
`onLock(data, ..., ...)` -> `pending[msg.sender][dataHash]++`
`onRelease(data, ..., ..., ...)` -> `eas.attest(...)`
and cause the hook to issue a validation attestation for any chosen `attestationUid`, recipient, expiry, and revocability settings.

This breaks the core meaning of attestations emitted by these hooks: downstream arbiters, escrows, or off-chain consumers can be tricked into accepting hook-issued attestations that were never produced by an actual escrow release. A user can therefore fabricate fulfillment/validation evidence from the hook's own attester identity and use it anywhere that identity is trusted.

Severity Note:
- Some integrators or off-chain actors treat attestations from these hook contracts' attester identity as authoritative evidence of escrow fulfillment without verifying provenance beyond the attester address.

13 of 18 Findings

contracts/src/utils/atomic/AtomicPaymentUtils.sol

`permitAndPayBundleAndCollect` cannot handle bundles that repeat the same ERC20 token

Low Risk

`permitAndPayBundleAndCollect` executes one EIP-2612 permit per ERC20 entry in the bundle, then pulls the ERC20 items from the caller inside `_pullPaymentBundleTokens`. When the same ERC20 token address appears multiple times in `data.erc20Tokens`, `_permitBundle` overwrites the allowance for that token each time instead of accumulating it:

```
for (uint256 i = 0; i < data.erc20Tokens.length; i++) {
    _permitErc20(
        data.erc20Tokens[i],
        data.erc20Amounts[i],
        permits[i].deadline,
        permits[i].v,
        permits[i].r,
        permits[i].s
    );
}
```

After this loop, the helper only has the allowance set by the **last** permit for each repeated token. `_pullPaymentBundleTokens` then attempts multiple `safeTransferFrom` calls for that same token:

```
for (uint256 i = 0; i < data.erc20Tokens.length; i++) {
    IERC20(data.erc20Tokens[i]).safeTransferFrom(msg.sender, address(this),
data.erc20Amounts[i]);
}
```

For a bundle such as `[USDC, USDC]` with amounts `[10, 20]`, the second permit overwrites the first, so the helper starts with an allowance of `20`, not `30`. The first transfer consumes part of that allowance, and the later transfer reverts. This makes `permitAndPayBundleAndCollect` unusable for otherwise valid bundle demands that contain repeated ERC20 addresses.

14 of 18 Findings

contracts/src/utils/atomic/AtomicPaymentUtils.sol

Front-Running of ERC20 Permits Causes Denial of Service in Atomic Payment Flow

Low Risk

The `AtomicPaymentUtils` contract includes convenience functions (`permitAndPayWithErc20`, `permitAndPayErc20AndCollect`, and `permitAndPayBundleAndCollect`) that allow fulfillers to approve ERC20 token transfers and execute escrow collections in a single transaction via EIP-2612 permits.

These functions rely on the internal `_permitErc20` helper, which executes the permit call inside a `try-catch` block. If the permit call fails, the `catch` block explicitly reverts the transaction with a `PermitFailed` error.

Because the transaction's payload (including the permit signature `v, r, s`) is visible in the public mempool before it is mined, a malicious actor or MEV bot can extract these parameters and submit a direct call to the ERC20 token's `permit` function. The attacker's transaction will succeed, consuming the nonce and granting the required allowance to the `AtomicPaymentUtils` contract.

However, when the user's original transaction is subsequently executed, the `IERC20Permit(token).permit` call will revert because the signature's nonce has already been used. The `try-catch` block will catch this failure and revert the entire transaction. This allows an attacker to repeatedly cause the user's atomic payment and collection transactions to fail at no cost other than gas, resulting in a griefing Denial of Service (DoS).

15 of 18 Findings

contracts/src/obligations/payment/ERC1155PaymentObligation.sol

contracts/src/obligations/payment/TokenBundlePaymentObligation.sol

contracts/src/obligations/escrow/default/ERC1155EscrowObligation.sol

contracts/src/obligations/escrow/default/TokenBundleEscrowObligation.sol

contracts/src/obligations/escrow/unconditional/UnconditionalERC1155EscrowObligation.sol

contracts/src/obligations/escrow/unconditional/UnconditionalTokenBundleEscrowObligation.sol

contracts/src/obligations/escrow/hook-based/hooks/ERC1155EscrowHook.sol

contracts/src/utils/splitters/ERC1155Splitter.sol

contracts/src/utils/splitters/TokenBundleSplitterBase.sol

Multiple ERC1155 settlement paths treat a non-reverting `safeTransferFrom` as proof of payment or release

Low Risk

Several outbound or payment-side ERC1155 transfer paths only check whether `safeTransferFrom` reverted. They do not verify that balances actually changed after the call.

Representative examples:

```
// ERC1155PaymentObligation
try IERC1155(obligationData.token)
    .safeTransferFrom(payer, obligationData.payee, obligationData.tokenId,
obligationData.amount, "") {
} catch {
    revert ERC1155TransferFailed(...);
}
```

```
// ERC1155EscrowObligation
try IERC1155(decoded.token).safeTransferFrom(address(this), to, decoded.tokenId,
decoded.amount, "") {
} catch {
    revert ERC1155TransferFailed(...);
}
```

```
// ERC1155EscrowHook
try IERC1155(decoded.token).safeTransferFrom(address(this), to, decoded.tokenId,
decoded.amount, "") {}
catch {
    revert ERC1155TransferFailed(...);
}
```

The same "non-revert means success" pattern appears in the ERC1155 legs of `TokenBundlePaymentObligation`, `TokenBundleEscrowObligation`, `UnconditionalTokenBundleEscrowObligation`, `ERC1155Splitter`, and `TokenBundleSplitterBase`.

A malicious or non-compliant ERC1155 contract can satisfy this check by returning from `safeTransferFrom` without updating balances as expected. In that case:
- `ERC1155PaymentObligation` can mint a payment attestation even though the payee never received the tokens,
- ERC1155 escrows can be marked collected or reclaimed while the tokens remain stuck in the escrow/hook contract,
- splitter distribution functions can emit success while recipients never receive their assigned ERC1155 amounts.

This is especially notable because some inbound ERC1155 paths in the same codebase do perform balance-delta verification on lock, so the protocol is already relying on exact transfer semantics. The missing verification on outbound/payment legs breaks the expected asset-delivery guarantee whenever the token contract is not fully honest.

Severity Note:
- The ERC1155 token can return from safeTransferFrom without updating balances.
- Participants (escrow demand creators, payees, or split oracles) accept or interact with such a token address.
- The malicious/non-compliant token's balanceOf can be inconsistent with actual state (or collude to pass inbound delta checks).

16 of 18 Findings contracts/src/obligations/payment/TokenBundlePaymentObligation.sol

TokenBundlePaymentObligation does not verify ERC1155 legs, allowing bundle attestations with fake ERC1155 components

Low Risk

`TokenBundlePaymentObligation.transferBundle` validates ERC20 and ERC721 components, but its ERC1155 handling is only a raw call:

```
IERC1155(data.erc1155Tokens[i]).safeTransferFrom(from, data.payee, data.erc1155TokenIds[i], data.erc1155A
```

There is no code-existence check, no `try/catch`, and no post-transfer balance verification for the ERC1155 legs. A bundle can therefore specify an ERC1155 component whose address is an EOA or a contract with a non-reverting fallback. The call returns successfully, no ERC1155 asset moves, and execution continues until the bundle payment attestation is emitted.

Because the contract's `check` function later relies only on the attested bundle data, downstream consumers cannot distinguish these fabricated ERC1155 components from genuine transferred assets. This lets an attacker create bundle payment attestations that omit some or all promised ERC1155 consideration while still appearing valid at the attestation layer.

Severity Note:
- Escrow/integrators rely on check() output to release assets without additional on-chain verification of ERC1155 transfers.
- Demands can be misconfigured to include non-ERC1155 addresses, and such misconfigurations are not filtered elsewhere.

17 of 18 Findings | contracts/src/obligations/escrow/unconditional/UnconditionalTokenBundleEscrowObligation.sol

Broken access control in `unsafePartiallyCollectEscrow` combined with flexible array checks allows asset theft via poison tokens

Info

The `unsafePartiallyCollectEscrow` function is designed to allow a fulfiller to partially collect a token bundle escrow when some token transfers fail (e.g. due to blacklisting or revert-on-transfer tokens). However, it contains an incorrect access control check: `if (msg.sender != escrow.recipient) revert UnauthorizedCall();`. The `escrow.recipient` is the creator of the escrow (the payer), whereas the entity collecting the escrow is the fulfiller (`fulfillment.recipient`). This flaw prevents the fulfiller from executing a partial collection.

Simultaneously, the `check` function in `UnconditionalTokenBundleEscrowObligation` permits a fulfillment bundle to contain more tokens than demanded (e.g. `payment.erc20Tokens.length >= demand.erc20Tokens.length`), as it only validates the prefixes of the token arrays.

An attacker can exploit these two flaws to steal assets without paying. If Alice creates an escrow demanding 100 USDC via a bundle escrow, Bob (the attacker) can fulfill it by providing an escrow bundle containing 100 USDC plus an extra "poison" token designed to revert upon transfer. Because the prefix matches, `check` returns true, allowing Bob to successfully collect Alice's assets.
When Alice attempts to collect Bob's bundle escrow to receive her 100 USDC, the normal `collect` function will revert entirely because the poison token's transfer fails. Alice is forced to rely on `unsafePartiallyCollectEscrow`, but her call will revert because she is not the `escrow.recipient` (Bob is). As a result, Alice's funds are permanently trapped. Once the escrow expires, Bob can call `reclaim` or `unsafePartiallyReclaimExpired` to retrieve his 100 USDC, effectively stealing Alice's assets for free.

Severity Note:
- The standard (atomic) collect path in the base contract follows the same authorization pattern as the unsafe partial path (i.e., callable only by escrow.recipient).

18 of 18 Findings

contracts/src/obligations/CommitRevealObligation.sol

CommitRevealObligation owner can retroactively confiscate all outstanding bonds by shrinking the reveal window and redirecting slashed funds

Info

Outstanding commit bonds are governed by two owner-controlled variables that are read at reveal and slashing time, not snapshotted when the commitment is created:

```
uint256 public commitDeadline;
address public slashedBondRecipient;

function setCommitDeadline(uint256 _commitDeadline) external onlyOwner {
    commitDeadline = _commitDeadline;
}

function setSlashedBondRecipient(address _slashedBondRecipient) external onlyOwner {
    slashedBondRecipient = _slashedBondRecipient;
}
```

Reveal validity and slashing both use the current `commitDeadline` value:

```
if (block.timestamp > uint256(info.commitTimestamp) + commitDeadline) {
    revert RevealTooLate(revealedCommitment);
}
```

```
if (block.timestamp <= info.commitTimestamp + commitDeadline) {
    revert CommitDeadlineNotReached(commitment);
}
...
(bool success,) = slashedBondRecipient.call{value: amount}("");
```

Because the deadline is not fixed per commitment, the owner can wait until users have already locked ETH bonds with `commit`, then change `slashedBondRecipient` to an owner-controlled address and reduce `commitDeadline` to `0` or another very small value. At that point, every previously open commitment immediately becomes slashable even if it was still inside the deadline that existed when the user committed. Any address can then call `slashBond`, and the ETH is transferred to the owner-chosen recipient. The same parameter change also makes honest reveals fail via `RevealTooLate` under the new rules.

This is not just an admin pause or configuration toggle: it gives the owner a direct path to seize all live commitment bonds after users have already locked value under different assumptions.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.]
No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.